Phase 2 user flow: controlled local AI

Phase 2 adds an optional local-model gateway to the existing static and runtime evidence assistants. It also lets authorized users ask a local model to select relevant, explicitly granted source ranges. This is evidence selection, not autonomous code generation or modernization execution.

What is available

- The existing deterministic assistants continue to work without a model.
- An approved local model can select relevant facts from a small server-built context. ArchIntel validates every returned citation and displays the corresponding stored evidence, never unrestricted generated prose.
- Source requests select literal files and bounded line ranges through the existing Code Security Gateway. Suspected secrets block model dispatch; recognized sensitive identifiers are redacted or blocked by policy.
- Model authorization, rejection and completion are recorded in workspace audit and invocation history. Prompts, source text and raw model replies are not stored there.

Enable the prepared local model

Sign in as a workspace security administrator and open **Security & access**.

1. In **People and service identities**, find your account and select **Allow local model** if MODEL_USE_LOCAL is not already listed. Grant it separately to other permitted users. Existing accounts are not silently upgraded.
2. In **AI gateway**, register a provider with:
3. Provider name: a label such as `Local Ollama`.
4. Local model endpoint: `http://127.0.0.1:11434`.
5. Installed model name: `qwen2.5:0.5b`.
6. Select **Register local model**. Registration checks the installed model and records its digest. Then select **Check connection**.
7. Select it under **Approved model**. Review context bytes, calls per hour and sensitive-data handling. Choose **Enable local model policy**.
8. Return to analysis, open a saved result, and ask the Architecture assistant a supported question. Expand **Sources and limitations** to inspect the actual mode and gateway reason.

Ollama and the small qwen2.5:0.5b model were prepared locally during Phase 2 validation. On another machine, install Ollama and obtain the model through your approved software process. Model download requires network access; inference uses the installed local artifact. The ArchIntel launcher does not manage Ollama. Start the Ollama application if its endpoint is unavailable.

The same workspace policy applies to the runtime assistant. Its selected facts still come from the selected runtime session and retain their evidence links. Static/runtime observations and heuristic scores are not changed by the model.

Understand modes and fallback

Mode/reason	Meaning
LOCAL_MODEL_ASSISTED / VALIDATED_CITATIONS	A permitted local model returned valid IDs from the supplied evidence
LOCAL_EVIDENCE / MODEL_DISABLED	Workspace policy is off; deterministic evidence is used

Mode/reason	Meaning
MODEL_PERMISSION_REQUIRED	This identity lacks MODEL_USE_LOCAL or the relevant evidence/source permission
PROVIDER_UNAVAILABLE / MODEL_NOT_INSTALLED	The registered local service/model cannot be used; no external fallback occurs
MODEL_REVISION_CHANGED	The installed model digest changed; review and register the new revision
SECRET_DETECTED / PROMPT_INJECTION	Filtering blocked the request before chat dispatch
SENSITIVE_DATA	Policy blocks recognized sensitive identifiers
CONTEXT_LIMIT / HOURLY_BUDGET	The bounded context or workspace attempt budget was exceeded
INVALID_RESPONSE / INVALID_CITATIONS / MODEL_ABSTAINED	The reply was rejected or supplied no usable citation; deterministic evidence is used
AUTHORIZATION_CHANGED / SOURCE_SCOPE_REVOKED	Policy/scope changed; model output is not released
FILTER_OR_PROVIDER_FAILURE	Filtering or the provider failed; no raw error details or model output are exposed

For source queries, a blocked/failed model request returns **no source facts**. The existing separately authorized local source-read operation remains available. Do not interpret an empty AI result as evidence that the source has no relevant behavior.

Ask about source without sharing a repository

1. Register a commit-pinned repository and grant explicit file/directory scope as described in PHASE-1-USER-FLOW.md.
2. Grant the caller both SOURCE_READ and MODEL_USE_LOCAL. Enable the chosen local-model policy.
3. Under **Ask about an authorized source range**, enter the repository ID, exact registered commit, literal filename, start/end lines, question and a non-sensitive purpose.
4. Choose **Query source range**. The UI requests one range; the API supports up to ten ranges, no more than 80 lines per range and 200 total lines, within the existing file/byte limits and model-context budget.
5. Review selected range citations and filtered excerpts. Clear the displayed response when finished.

Only requested ranges enter the model envelope. The gateway reads the authorized committed blobs to extract those ranges; unselected lines are not sent. Source path/commit/scope must still be valid when dispatch is authorized and before model output is returned. No file is modified and no generated command is executed.

Manage policy and inspect audit

Disable model policy stops new model authorizations and leaves deterministic assistants available. **Disable provider** prevents using that registration. Revoking MODEL_USE_LOCAL prevents future authorized calls for that identity. A request already sent to the local model cannot be unsent; the gateway rechecks permissions/policy before releasing its output.

The policy defaults are: model disabled, external transfer DENY, telemetry DISABLED, 16,384 context bytes, 128 output tokens and 30 model attempts per workspace per hour. Failed attempts after authorization consume budget. Output-token limits are available through the policy API (32-512); UI controls cover context and hourly budget.

Model invocation history shows the latest 100 workspace requests, outcome/reason, context size and redaction categories. The audit trail contains correlated request IDs, policy revision, context digest, local model digest and usage counts when reported by the provider. Currency cost is unknown for local inference, not reported as zero.

Limits and troubleshooting

- Only operator-allowlisted literal loopback HTTP endpoints are supported. External, enterprise-remote and private-cloud adapters are not enabled in this delivery. There is no external fallback.
- Secret/PII and injection detection are heuristic. Recognized identifiers include email, phone, payment-like numbers and US SSN format. This is not comprehensive name/address/health-data classification or a compliance certification.
- Small local models may select poor or empty citations. Check the evidence; the model does not prove business ownership, architecture correctness or modernization safety.
- If a model is updated, register it again and explicitly select its new registration. Existing registrations are immutable except for enable/disable.
- Old LLM_URL/LLM_MODEL settings do not authorize calls. Use provider registration, policy and explicit permissions.
- Keep this deployment on localhost. The gateway constrains ArchIntel HTTP dispatch; it does not install host firewall rules or control every outbound connection of an independently managed model service. Operators must enforce the model host's own egress/logging policy for an air-gapped deployment.

Review PHASE-2-VERIFICATION.md and the developer handoff before approving the next phase. Phase 3 runtime foundation work has not started as part of this delivery.